

Você recebeu o material necessário para realizar a segunda avaliação. São dez atividades práticas que devem ser desenvolvidas no computador. Enviar somente o arquivo **prova_rick_morty.py**. Não altere o nome do arquivo ou o nome das funções.

Exercício 1 — Contar espécies únicas

Implemente uma função chamada `contar_especies` que recebe um dicionário no formato da resposta da API pública da série Rick and Morty. Esse dicionário contém uma chave chamada "results", que armazena uma lista de personagens, cada um representado por um dicionário com diversas informações.

A função deve percorrer a lista de personagens e contabilizar quantos personagens pertencem a cada espécie. O resultado deve ser retornado como um dicionário onde cada chave é o nome de uma espécie e o valor correspondente é a quantidade de personagens daquela espécie encontrados na lista.

Exemplo 1

Entrada:

```
{"results": [  
  {"name": "Rick",    "species": "Human"},  
  {"name": "Morty",   "species": "Human"},  
  {"name": "Birdperson", "species": "Bird-Person"}  
]}
```

Saída:

```
{"Human": 2, "Bird-Person": 1}
```

Exemplo 2

Entrada:

```
{"results": [  
  {"name": "Mr. Meeseeks", "species": "Alien"},  
  {"name": "Squanchy",    "species": "Cat-Person"},  
  {"name": "Alien",       "species": "Alien"}  
]}
```

Saída:

```
{"Alien": 2, "Cat-Person": 1}
```

Exercício 2 — Filtrar personagens por status

Implemente uma função chamada `filtrar_por_status` que recebe dois parâmetros: um dicionário no formato da resposta da API pública da série Rick and Morty e uma string representando o status desejado. O dicionário contém uma chave chamada "results", que armazena uma lista de personagens, cada um representado por um dicionário com diversas informações.

A função deve percorrer a lista de personagens e selecionar apenas aqueles cujo campo "status" seja igual ao valor recebido como parâmetro. Os status possíveis na API são "Alive", "Dead" e "unknown". O resultado deve ser retornado como uma lista contendo apenas os **nomes** dos personagens que correspondem ao status informado, na mesma ordem em que aparecem na entrada.

Exemplo 1

Entrada:

```
dados = {"results": [  
    {"name": "Rick",      "status": "Alive"},  
    {"name": "Beth",      "status": "Alive"},  
    {"name": "Birdperson", "status": "Dead"}  
]}
```

status = "Alive"

Saída:

```
["Rick", "Beth"]
```

Exemplo 2

Entrada:

```
dados = {"results": [  
    {"name": "Rick",      "status": "Alive"},  
    {"name": "Birdperson", "status": "Dead"},  
    {"name": "Noob Noob",  "status": "unknown"}  
]}
```

status = "Dead"

Saída:

```
["Birdperson"]
```

Exercício 3 — Extrair nomes em maiúsculo

Implemente uma função chamada `nomes_maiusculo` que recebe um dicionário no formato da resposta da API pública da série Rick and Morty. Esse dicionário contém uma chave chamada "results", que armazena uma lista de personagens, cada um representado por um dicionário com diversas informações.

A função deve percorrer a lista de personagens, extrair o nome de cada um e convertê-lo para letras maiúsculas. O resultado deve ser retornado como uma lista contendo todos os nomes convertidos, na mesma ordem em que aparecem na entrada.

Exemplo 1

Entrada:

```
{"results": [  
  {"name": "Rick Sanchez"},  
  {"name": "Morty Smith"}  
]}
```

Saída:

```
["RICK SANCHEZ", "MORTY SMITH"]
```

Exemplo 2

Entrada:

```
{"results": [  
  {"name": "Beth Smith"},  
  {"name": "Jerry Smith"},  
  {"name": "Summer Smith"}  
]}
```

Saída:

```
["BETH SMITH", "JERRY SMITH", "SUMMER SMITH"]
```

Exercício 4 — Agrupar personagens por gênero

Implemente uma função chamada `agrupar_por_genero` que recebe um dicionário no formato da resposta da API pública da série Rick and Morty. Esse dicionário contém uma chave chamada "results", que armazena uma lista de personagens, cada um representado por um dicionário com diversas informações.

A função deve percorrer a lista de personagens e agrupá-los de acordo com o campo "gender". Os gêneros possíveis na API são "Male", "Female", "Genderless" e "unknown". O resultado deve ser retornado como um dicionário onde cada chave é um gênero encontrado na lista e o valor correspondente é uma lista com os nomes dos personagens que pertencem àquele gênero, na mesma ordem em que aparecem na entrada.

Exemplo 1

Entrada:

```
{"results": [  
  {"name": "Rick",    "gender": "Male"},  
  {"name": "Summer", "gender": "Female"},  
  {"name": "Morty",   "gender": "Male"}  
]}
```

Saída:

```
{"Male": ["Rick", "Morty"], "Female": ["Summer"]}
```

Exemplo 2

Entrada:

```
{"results": [  
  {"name": "Beth",    "gender": "Female"},  
  {"name": "Alien X", "gender": "unknown"},  
  {"name": "Jerry",   "gender": "Male"}  
]}
```

Saída:

```
{"Female": ["Beth"], "unknown": ["Alien X"], "Male": ["Jerry"]}
```

Exercício 5 — Calcular média dos IDs

Implemente uma função chamada `media_ids` que recebe um dicionário no formato da resposta da API pública da série Rick and Morty. Esse dicionário contém uma chave chamada "results", que armazena uma lista de personagens, cada um representado por um dicionário com diversas informações.

A função deve percorrer a lista de personagens, coletar o valor do campo "id" de cada um e calcular a média aritmética desses valores. O resultado deve ser retornado como um número de ponto flutuante (float).

Exemplo 1

Entrada:

```
{"results": [  
  {"id": 1, "name": "Rick"},  
  {"id": 2, "name": "Morty"},  
  {"id": 3, "name": "Summer"}  
]}
```

Saída:

2.0

Exemplo 2

Entrada:

```
{"results": [  
  {"id": 10, "name": "Beth"},  
  {"id": 20, "name": "Jerry"}  
]}
```

Saída:

15.0

Exercício 6 — Listar espécies sem repetição

Implemente uma função chamada `especies_unicas` que recebe um dicionário no formato da resposta da API pública da série Rick and Morty. Esse dicionário contém uma chave chamada "results", que armazena uma lista de personagens, cada um representado por um dicionário com diversas informações.

A função deve percorrer a lista de personagens, coletar o valor do campo "species" de cada um e eliminar as repetições. Para isso, utilize um conjunto (set) durante o processamento. O resultado deve ser retornado como uma lista contendo cada espécie apenas uma vez, ordenada em ordem alfabética.

Exemplo 1

Entrada:

```
{"results": [  
    {"species": "Human"},  
    {"species": "Alien"},  
    {"species": "Human"},  
    {"species": "Robot"}  
]}
```

Saída:

```
["Alien", "Human", "Robot"]
```

Exemplo 2

Entrada:

```
{"results": [  
    {"species": "Cronenberg"},  
    {"species": "Human"},  
    {"species": "Cronenberg"}  
]}
```

Saída:

```
["Cronenberg", "Human"]
```

Exercício 7 — Criar tuplas de identificação

Implemente uma função chamada `tuplas_personagens` que recebe um dicionário no formato da resposta da API pública da série Rick and Morty. Esse dicionário contém uma chave chamada "results", que armazena uma lista de personagens, cada um representado por um dicionário com diversas informações.

A função deve percorrer a lista de personagens e, para cada um, criar uma tupla contendo exatamente três elementos na seguinte ordem: o valor do campo "id", o valor do campo "name" e o valor do campo "status". Todas as tuplas geradas devem ser adicionadas a uma lista, que deve ser retornada ao final da execução.

Exemplo 1

Entrada:

```
{"results": [  
    {"id": 1, "name": "Rick", "status": "Alive"},  
    {"id": 2, "name": "Morty", "status": "Alive"}  
]}
```

Saída:

```
[(1, "Rick", "Alive"), (2, "Morty", "Alive")]
```

Exemplo 2

Entrada:

```
{"results": [  
    {"id": 5, "name": "Birdperson", "status": "Dead"}  
]}
```

Saída:

```
[(5, "Birdperson", "Dead")]
```

Exercício 8 — Verificar se um personagem existe

Implemente uma função chamada `personagem_existe` que recebe dois parâmetros: um dicionário no formato da resposta da API pública da série Rick and Morty e uma string contendo o nome de um personagem a ser buscado. O dicionário contém uma chave chamada "results", que armazena uma lista de personagens, cada um representado por um dicionário com diversas informações.

A função deve percorrer a lista de personagens e verificar se algum deles possui o campo "name" igual ao nome recebido como parâmetro. A comparação deve ignorar diferenças entre letras maiúsculas e minúsculas. Caso o personagem seja encontrado, a função deve retornar `True`; caso contrário, deve retornar `False`.

Exemplo 1

Entrada:

```
dados = {"results": [  
    {"name": "Rick Sanchez"},  
    {"name": "Morty Smith"}  
]}  
  
nome = "morty smith"
```

Saída:

`True`

Exemplo 2

Entrada:

```
dados = {"results": [  
    {"name": "Rick Sanchez"},  
    {"name": "Morty Smith"}  
]}  
  
nome = "Jerry Smith"
```

Saída:

`False`

Exercício 9 — Indexar personagens por ID

Implemente uma função chamada `indexar_por_id` que recebe um dicionário no formato da resposta da API pública da série Rick and Morty. Esse dicionário contém uma chave chamada "results", que armazena uma lista de personagens, cada um representado por um dicionário com diversas informações.

A função deve percorrer a lista de personagens e construir um novo dicionário onde cada chave é o valor do campo "id" do personagem (número inteiro) e o valor correspondente é um dicionário contendo apenas dois campos: "nome", com o valor do campo "name", e "status", com o valor do campo "status". Os demais campos do personagem devem ser descartados. Esse dicionário deve ser retornado ao final da execução.

Exemplo 1

Entrada:

```
{"results": [  
  {"id": 1, "name": "Rick", "status": "Alive", "species":  
"Human"},  
  {"id": 2, "name": "Morty", "status": "Alive", "species":  
"Human"}  
]}
```

Saída:

```
{  
  1: {"nome": "Rick", "status": "Alive"},  
  2: {"nome": "Morty", "status": "Alive"}  
}
```

Exemplo 2

Entrada:

```
{"results": [  
  {"id": 5, "name": "Birdperson", "status": "Dead", "species":  
"Bird-Person"}  
]}
```

Saída:

```
{5: {"nome": "Birdperson", "status": "Dead"}}
```

Exercício 10 — Gerar resumo estatístico

Implemente uma função chamada `resumo_estatistico` que recebe um dicionário no formato da resposta da API pública da série Rick and Morty. Esse dicionário contém uma chave chamada "results", que armazena uma lista de personagens, cada um representado por um dicionário com diversas informações.

A função deve percorrer a lista de personagens e calcular as seguintes informações: o número total de personagens na lista; a quantidade de espécies únicas presentes, utilizando um conjunto (set) durante o processamento; a quantidade de personagens com o campo "status" igual a "Alive"; e a quantidade de personagens com o campo "status" igual a "Dead". O resultado deve ser retornado como um dicionário com as chaves "total", "especies", "vivos" e "mortos", nessa ordem.

Exemplo 1

Entrada:

```
{"results": [  
  {"status": "Alive", "species": "Human"},  
  {"status": "Dead", "species": "Human"},  
  {"status": "Alive", "species": "Alien"}  
]}
```

Saída:

```
{"total": 3, "especies": 2, "vivos": 2, "mortos": 1}
```

Exemplo 2

Entrada:

```
{"results": [  
  {"status": "Dead", "species": "Robot"},  
  {"status": "Dead", "species": "Robot"},  
  {"status": "unknown", "species": "Alien"}  
]}
```

Saída:

```
{"total": 3, "especies": 2, "vivos": 0, "mortos": 2}
```